

分工 副本

🌟 总览

- **张瑞喆**：把控全局基建，负责多智能体共识与自愈引擎。
- **林子豪**：负责 RAG 检索与大模型底层调用。
- **成可心 (前端负责人) + 张钊 (算法兼前端)**：用户交互与感知
- **胡曦元 (后端负责人) + 杨钞越 (算法兼后端)**：基础设施与编排

🔧 具体人员职责拆解

1. 张瑞喆 (技术主负责人 Tech Lead) —— 架构、规范与核心攻坚

聚焦标准制定和算法实现。

- **标准制定 (30%)**:
 - 确定前后端 API 契约（定义清晰的 JSON Schema，让 成可心 和 胡曦元 尽早解耦并行）。
 - 统筹所有 Agent 的 Prompt 模板规范（集中管理，防止其他人乱写）。
 - 代码 Review（特别是大模型相关的边界条件处理）。
- **核心算法开发 (70%)**:
 - **实现阶段四：Multi-Agent 状态机 (LangGraph)**。这是系统大脑，编写 Builder、Reviewer、Security 三者的共识路由网络。
 - **实现阶段三：并行自愈引擎 (Chaos Engineering)**。处理复杂的异步高并发（`asyncio.gather`），攻克沙箱竞速和资源熔断的底层逻辑。

2. 林子豪 (算法工程师) —— RAG 与底层模型基建

解决“找得准”和“读得懂”的问题。

- **核心算法开发 (100%)**:
 - **实现阶段一：意图结构化与双轨召回**。打通 LLM 提取 JSON，对接 GitHub 搜索 API（维护 Token 轮询池），建立 BM25 索引。
 - **打分与重排引擎**：编写粗排公式和 LLM 精排逻辑。
 - 封装统一的 LLM Client（对接 OpenAI/Claude），处理 Rate Limit、重试和超时降级逻辑。

3. 胡曦元 (后端负责人) + 杨钊越 (算法兼后端) —— 基础设施

负责构建底座，把 张瑞喆 和 林子豪 写的算法包装成高可用的服务。

- **胡曦元 (后端主R):**
 - **基建与环境:** 搭建 FastAPI 框架，配置 PostgreSQL 数据库 (SQLAlchemy ORM)，设置 Docker 容器化。
 - **外部集成:** 对接 Sealos K8s API，编写部署实例的 CRUD 接口。
 - **分担的算法辅助:** 编写非 LLM 强相关的“脏活”算法。例如：阶段二中的“智能上下文截断器（通过正则提取依赖文件）”、“错误日志降噪清洗（截取 Error 上下文）”。
- **杨钊越 (算法主R，辅助后端):**
 - **核心算法开发:** 负责编写 **阶段二：评估与生成** 中的“降级生成机制（子路径1：Vibe 脚手架引擎，子路径2：代码缝合组件）”。
 - **后端辅助:** 因为 杨钊越 写的降级机制非常耗时，杨钊越 需要自己把这部分逻辑包装进 **Celery 异步任务队列** 中，并使用 Redis 维护任务状态，减轻 胡曦元 的后排压力。

4. 成可心 (前端负责人) + 张钊 (算法兼前端) —— 用户交互

在 UI 暂时未交付的情况下，重点是“跑通流转链路”和“状态机对接”。

- **成可心 (前端主R):**
 - **基建与占位:** 初始化 Expo / React Native 项目，搭建全局路由（导航）。
 - **素颜 UI 开发:** 用原生的基础组件拼凑出一个“能用”的 Chatbot 界面和 App Gallery 列表（不追求美观，只追求数据能绑上去）。
 - **分担的算法辅助:** 因为 UI 没出，**成可心** 负责“**展现层相关的 Prompt Engineering**”。例如：调试出能够稳定生成适配前端 Markdown 渲染、卡片 JSON 结构的提示词；配合大模型做返回结果的格式化校验。
 - **调研任务：常见构建报错与修复方案库**
 - **爬取目标:** StackOverflow (标签: docker, kubernetes, deployment)、GitHub Issues 中带有 `build failed` 标签的热门讨论。
 - **目的:** 当“自愈 Agent”遇到报错时，不应该让它盲猜，而是通过 RAG 检索真实的解决方案。例如：爬取 `npm ERR! gyp ERR!` 对应的最佳修复方案（通常是补齐 python 和 build-essential），将其固化为知识库。
- **张钊 (算法主R，辅助前端):**

- **核心算法开发**：负责编写 **阶段四中的具体子 Agent**，比如负责具体干活的“Agent 1 (Builder)”和负责查漏补缺的“Agent 2 (Reviewer)”，并将写好的节点接入 张瑞喆 设计的图 (Graph) 中。
 - **前端辅助**：因为 张钊 非常清楚 Agent 的流转状态 (Thinking -> Building -> Healing)，张钊 负责协助 成可心 联调 **WebSocket 长连接**，将后端的流式日志和状态机变更精准地推送到前端页面上。
 - **调研任务：黄金基础镜像映射表**
 - **调研目标**：Docker Hub 官方镜像的最佳实践。
 - **目的**：大模型经常会瞎编基础镜像（比如用到非常臃肿的 `ubuntu:latest`）。你需要调研并整理一份映射表：`Node.js -> node:18-alpine`，`Python+PyTorch -> pytorch/pytorch:2.0.1-cuda11.7-cudnn8-runtime`。在生成 Dockerfile 时强制 Agent 从这个白名单里选。
-

项目推进建议

建议zrz按照以下里程碑 (Milestones) 来推进：

1. 首周核心任务 (API-First & Mock)：

- **绝对不要等算法跑通再联调！**
- zrz 带着大家花半天时间定死所有的 JSON 数据结构。
- 胡曦元 写出 FastAPI 的假接口 (Mock API)，成可心 立刻开始对着 Mock 数据写前端。

2. “伪”智能体联调期：

- 在 张瑞喆 写出复杂的 LangGraph、林子豪 写出精准的 RAG 之前，先用最简单的 `prompt + openai_call` 填补核心函数。
- 让整条链路（前端输入 -> 后端接收 -> 假算法返回 -> Sealos部署 -> 前端展示）在第二周就粗糙地跑起来。

3. 算法深水区攻坚：

- 当主干通了之后，林子豪 专心去优化召回率，张瑞喆 专心去写自愈防腐层。此时如果前端 UI 稿出来了，成可心 就能立刻切换回“装修”模式，张钊 和 杨钊越 也能逐步将算法代码向工程化代码迁移。